



CSE 332: Computer Architecture and Organization

Assignment#2 Spring 2025

Submitted by,

Muhtadina Serniabat Tasin

ID: 2232160642

Course: CSE332

Section: 5

Semester: Spring 2025

Submitted to,

Dr. Mohammad Abdul Qayum [MAQm]

Submission Date: 1 March, 2025

Due Date: 4 March, 2025

1. MIPS instructions:

1. Move content of register r11 into register r23.
2. Load register r15 with the value 265,140.
3. Store the value 55,259 into a word at memory location 40000.
4. The equation $a = b + c$, assuming a, b and c are integer variables at memory location 0b10000000, 0b1001000 and 0b100010011100, respectively.
5. The equation $y[5] = y[10] + x$, assuming x corresponds to register r2, and y[1] is an integer at the memory location 524,292 is the first element of vector y.
6. The statement: if (a <= b) go to lab1, assuming that variable a corresponds to the word at memory address 400, and variable b corresponds to register r10. The instruction with lab1 is before the branch instruction and there are 20 (machine) instructions between those two instructions.

Ans. 1.

```
move $r23, $r11 # Moving content of Register r11 into Register r23.
```

Ans. 2.

Value = 265,140

So, $(265140)_{10} = (40BB4)_{16}$

Upper 16 bits = $(0 \times 0004)_{16} = (4)_{10}$

Lower 16 bits = $(0 \times 0BB4)_{16} = (2996)_{10}$

```
lui $r15, 4           # Load Upper Immediate: (4 * 2^16 = 262144)
ori $r15, $r15, 2996  # OR Immediate: Add the lower 16-bit part
                      # (265140 - 262144 = 2996)
```

Ans. 3.

```
lui $t0, 0           # Load upper part of memory address
ori $t0, $t0, 40000  # Complete the address

li $t1, 55259        # Load value 55,259 into a temporary register
sw $t1, 0($t0)       # Store value at address 40,000
```

Ans. 4.

variable **a** at location = 0b10000000 = $(128)_{10}$

variable **b** at location = 0b1001000 = $(72)_{10}$

variable **c** at location = 0b100010011100 = $(1100)_{10}$

```
lw $t0, 72($zero)    # Load b from memory (address 72) into $t0
lw $t1, 1100($zero)  # Load c from memory (address 1100) into $t1
add $t2, $t0, $t1    # t2 = b + c
sw $t2, 128($zero)   # Store result in a (address 128)
```

Ans. 5.

y[1] at location = 524292

So, **y[i]** at = $52492 + (i-1) \times 4$

y[5] = $524292 + 16 = 524308$

y[10] = $524292 + 36 = 524328$

```
lw $t0, 524328($zero) # Load y[10] from memory at 524,328
add $t0, $t0, $r2      # y[10] + x
sw $t0, 524312($zero) # Store result in y[5] at 524,308
```

Ans. 6.

```
lw $t0, 400($zero)    # Load a from memory
ble $t0, $r10, lab1   # Branch if a <= b
```

2. Bit Pattern for each MIPS Instruction:

```
lap: lw r16, 0(13)
     addi r8, r2, -513
     nop
     sw r8, -18(r4)
     bne r3, r0, lap %lap=25
```

Instruction	Type	Opcode	rs	rt	rd/Offset	shamt	funct	Machine Code (Binary)	Machine Code (Hex)
lw r16, 0(13)	I-Type	100011	011 01	100 00	0000 0000 0000 0000	-	-	100011 01101 10000 00000000000000 000	0x8D1000 00
addi r8, r2, -513	I-Type	001000	000 10	010 00	1111 1110 0000 0000	-	-	001000 00010 01000 1111111000000 000	0x2048FE 00
nop (encoded as sll \$0, \$0, 0)	R-Type	000000	000 00	000 00	00000	00000	00000 0	000000 00000 00000 00000 00000 000000	0x000000 00
sw r8, -18(r4)	I-Type	101011	001 00	010 00	1111 1111 1110 1110	-	-	101011 00100 01000 1111111111101 110	0xA488FF EE
bne r3, r0, lap (25)	I-Type	000101	000 11	000 00	0000 0000 0001 1001	-	-	000101 00011 00000 00000000000011 001	0x146000 19

3. Assembly Program in MIPS32 ISA for the following C program:

```

/// C program

#define N 11
int ARRAY[N] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21};
void main (void)
{
    int total = sum_array(ARRAY);
    printf ("The total is: %d", total);
}
int sum_array(int ARRAY[N])
{
    int total=0;
    for (int i=0; i<N; i++ )
    {
        total+=ARRAY[i];
    }
    return total;
}

```

```

# MIPS32 ISA of the following C program

.data
    ARRAY: .word 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21
    # Define ARRAY with 11 elements
    N: .word 11 # Define N = 11
    message: .asciiz "The total is: " # String for printf output

.text
.globl main
main:
    # Load address of ARRAY into $a0
    la $a0, ARRAY

    # Call sum_array function
    jal sum_array

    # Store result in $s0 for printing
    move $s0, $v0

    # Print message "The total is: "
    li $v0, 4
    la $a0, message
    syscall

    # Print the sum
    li $v0, 1
    move $a0, $s0
    syscall

```

```

    # Exit program
    li $v0, 10
    syscall

sum_array:
    # Function prologue
    addi $sp, $sp, -8 # Allocate space on stack
    sw $ra, 4($sp)    # Save return address
    sw $s1, 0($sp)    # Save s1

    li $v0, 0 # total = 0
    li $s1, 0 # i = 0

loop:
    # Check if i < N
    li $t0, 11
    bge $s1, $t0, end_sum_array

    # Load ARRAY[i]
    sll $t1, $s1, 2 # i * 4 (word offset)
    add $t2, $a0, $t1 # Address of ARRAY[i]
    lw $t3, 0($t2) # Load ARRAY[i] into $t3

    # total += ARRAY[i]
    add $v0, $v0, $t3

    # Increment i
    addi $s1, $s1, 1
    j loop

end_sum_array:
    # Restore stack and return
    lw $ra, 4($sp)
    lw $s1, 0($sp)
    addi $sp, $sp, 8
    jr $ra

```